

TUTORIAL SESSIONS



5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 02:

SFML Tutorial #01

Introduction to the SEG SFML Engine

Tutorial Overview

This tutorial will introduce you to the Software Engineering for Games (SEG) engine framework, which is built on top of SFML 2.5.1, which you will use to development your game application as part of the assignment brief.

In this tutorial you will complete the following tasks:

- The **First Task** is read through the preliminary background and then to obtain and open the Visual Studio 2022 project for the 5210COMP SEG Engine.
- The **Second Task** is to explore the game engine, as it is the framework that you will use to develop your assignment.
- The **Third Task** is to create a blank game state.
- The **Fourth Task** is to add the game state to the game.
- The **Fifth Task** is to create a Text Game Object so you can show text on the screen.
- The **Sixth Task** is to add an instance of the Text Game Object to the game state.
- Additional tasks are for you to work on at home.

Preliminary Background

Background information on the framework:

GameState: A GameState in the SEG Engine works like the concept of a Level/Map in Unreal Engine or a scene in Unity – it is common in complex games to break your code into levels/sub-states.

- A GameState is essentially a level or a menu screen and so on. The engine switches between GameState to switch between menus and levels and so on. So, for example, if you wanted a main menu screen, you would create a main menu GameState that *derives from* the GameState base class.
- If you wanted to create a level, you would create a level GameState that *derives from* the GameState base class. **The GameState class maintains a list of GameObjects. This is the C++ concept of inheritance, which we will cover in the lectures.**

GameObject: GameObjects are the items that make up a GameState (or level). Everything within a GameState is considered a game object.

- For example, if you wanted to create a menu button, you would create a menu button GameObject or if you wanted to create a player, you would create a player GameObject and so on. This is the equivalent to the concept of an **Actor** in Unreal Engine.
- Once you have created an instance of GameObject you must initialise it and then push it onto the vector (C++ dynamic array) of GameObjects in the GameState. This will cause the **Initialise** and **Update** functions to be called automatically, which essentially loads the assets in the level, sets their initial values and then starts the game loop, which will contain the game logic and rendering code.

GameStateManager: The GameStateManager is responsible for controlling the GameStates. It uses a stack to store the game states that you push onto it.

- Pushing a GameState onto the GameState manager will cause the Game to switch to that new GameState. When you pop the GameState from the GameStateManager, the previous GameState will resume.
- For example, you could push a MainMenu GameState, then push a Level GameState, when the level is finished if you pop the level GameState the game will return to the MainMenu GameState.

Other Classes: There are many other classes in the framework that are used to create the overall structure of the project – one thing that is vital to get used to in games development is working with large complex codebases.

This is not a large code base (*I'd say medium-sized at best*), but it is certainly a step up from what you are used to at Level 4, so today, we are going to focus on:

- Understanding the Application structure
- Setting up the code base
- Effectively navigating it using Visual Studio as our Integrated Development Environment (IDE).

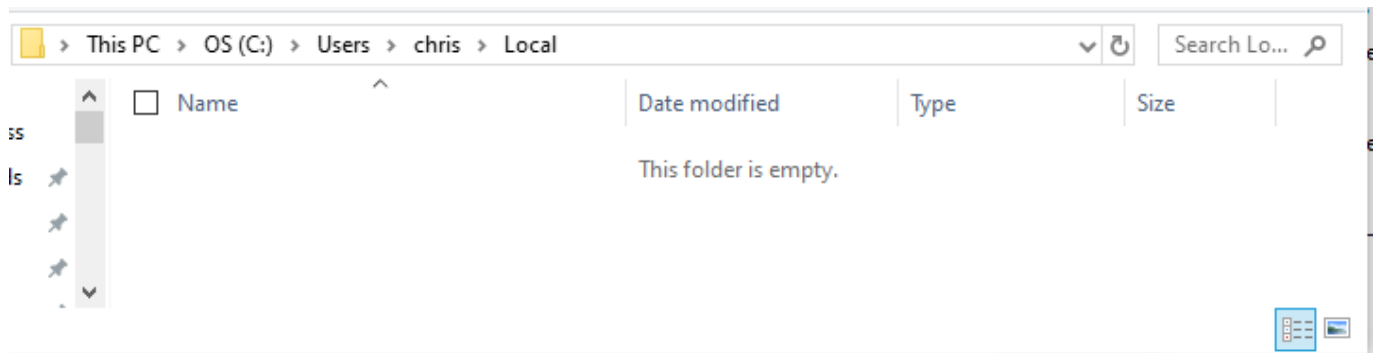
Task 1: Download the SEG Game Engine

Step 1: Create a new folder on your C: drive folder on your Lab Machine.

To make this easier, you should pick ONE machine in the Lab that you will be using in EVERY session on this module.

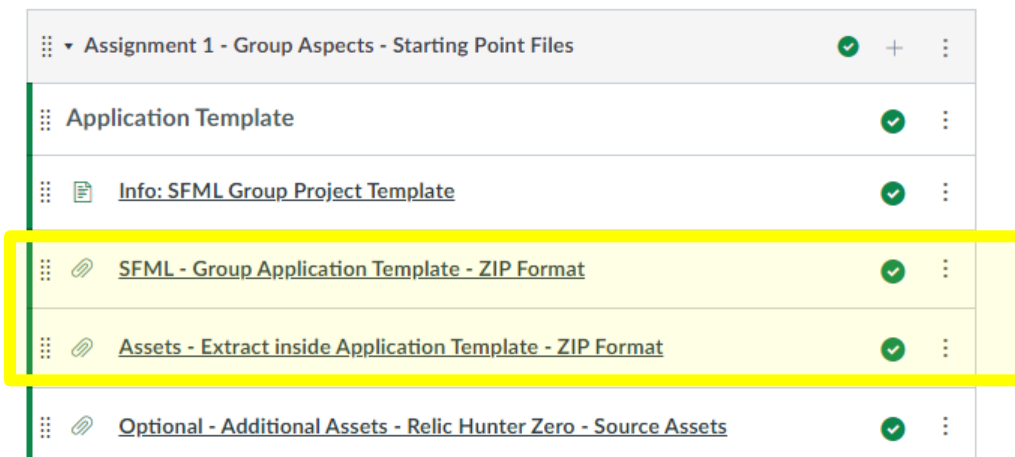
Create a new folder on your local profile area on the C: drive (e.g. **C:\users\<username>\Local**) is suggested, where <username> is your LJMU username – similar to the screenshot below).

This is where we will put the SEG Game Engine.



Step 2: Download the Two Required Zip file zip files

Download the following highlighted ZIP files from the module's Canvas site:



Move the downloaded zip files into your newly created folder – they should be the following:

SEG_Relic_Assets.zip

SEG_Relic_AppTemplate.zip

This PC > ARVRLab (C:) > Users > chris.CMPGL42 > Local				
Name	Date modified	Type	Size	
SEG_Rellic_AppTemplate.zip	23/07/2022 11:12	Compressed (zipp...	44,936 KB	
SEG_Rellic_Assets.zip	23/07/2022 11:28	Compressed (zipp...	490,002 KB	

As you can see, the largest files are not the code and its external libraries, it is the Assets.

In a lot of games projects, Assets make up over 90% of the sizes – textures, backgrounds, images, sprites and 3D models contain huge amounts of data, which we load into memory using our C++ code – the code itself to do this is not that big, so doesn't take up much physical storage space.

IMPORTANT NOTE: Your User Profile will stay active on a lab machine for 30 days since last login – therefore, on this module, you will be using the SAME MACHINE in each lab session. **Make a note of your machine name/number when you enter the lab.**

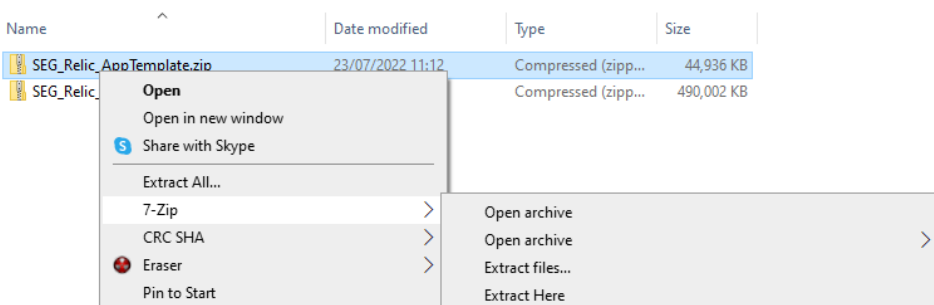
Once you have completed the lab sessions, it is highly recommended you back them up to your OneDrive storage as Zip Files

Data corruption will not be considered valid grounds for an extension – part of the software engineering lifecycle is maintaining your code base and backing/versioning data is a big part of this.

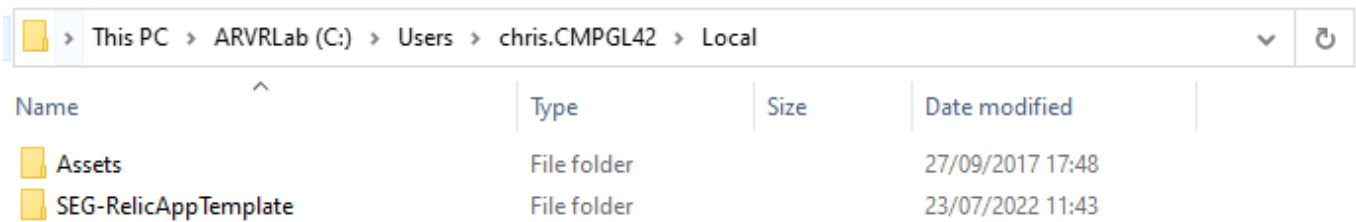
Step 3: Unzip the two ZIP files

Using 7-zip, unzip the files using the **'Extract Here'** option.

7-Zip is accessible in the Right-Click context menu, so just right click on each zip file.



Once they are extracted, you can delete the Zip files. You should end up with the following folders, as shown below.



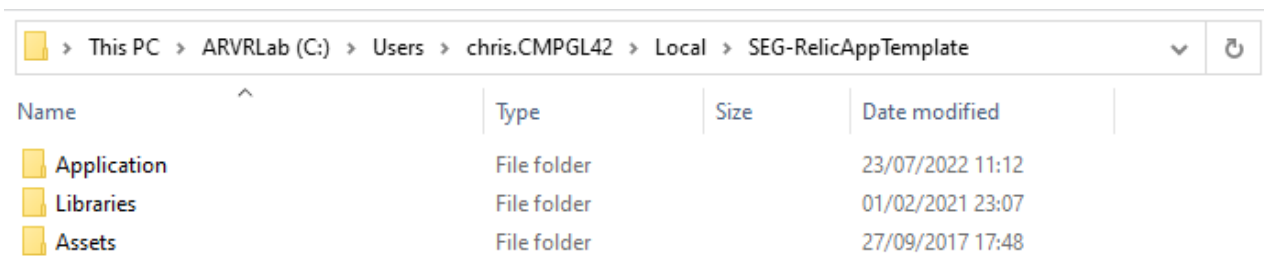
Name	Type	Size	Date modified
Assets	File folder		27/09/2017 17:48
SEG-RelicAppTemplate	File folder		23/07/2022 11:43

Step 4: Sorting out the Project Structure

These are the correct files, **but they are not the correct code structure** – our next task is to sort out the code structure, so that they are in the correct places, as all the paths must line up with what the C++ project configuration in Visual Studio expects.

Luckily, in this case, this is an easy job. Select the Assets folder in Windows Explorer and then *Cut* them.

Open the folder **SEG-RelicAppTemplate** and paste the folder into the root of this folder. You should have the following:



Name	Type	Size	Date modified
Application	File folder		23/07/2022 11:12
Libraries	File folder		01/02/2021 23:07
Assets	File folder		27/09/2017 17:48

This is our final structure for the Project, we have the following folders:

- **Application:** This contains our Engine Source Code and our Game code and is a Visual Studio 2022 Solution.
- **Libraries:** Contains our two external dependencies – these are SFML 2.5.1 and Tiny XML.
- **Assets:** Contains the raw assets which we will be using as the graphical and gameplay data in our game.

Although you were introduced to SFML with Sud in the second semester of Mathematics and 2D Graphics, this time, we will not be using it so much directly, but instead engineering a game framework and game on top of SFML.

This is much more common in the games industry – Unreal makes use of lots of different frameworks which it is built on top of like PhysX, FBX, Apex, Windows SDK, Android, AR Core, Oculus, DirectX, PlayStation 5 SDK etc.

We typically call these libraries **Middleware** – they are in-between our **Software** and our **Hardware**.

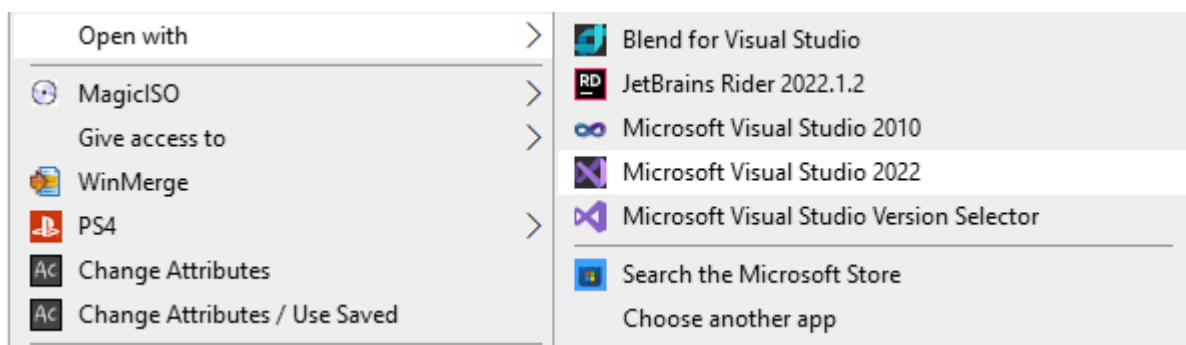
Task 2: Explore the SEG Game Engine

Step 1: Open the SEG engine in Visual Studio

Find the project solution inside the Application folder (.sln) file and open it using Visual Studio 2022.

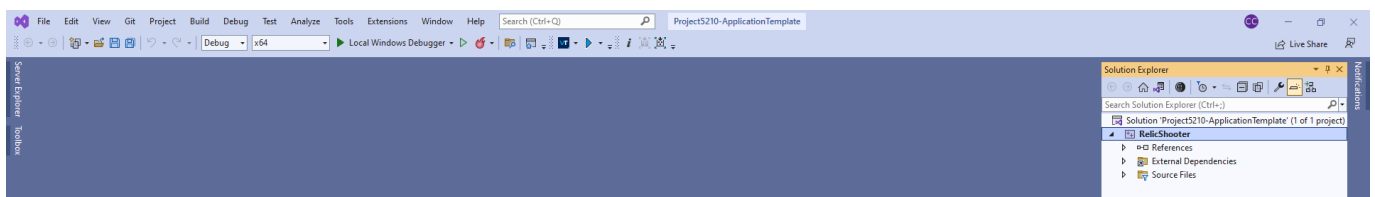
Name	Date modified	Type	Size
 RelicShooter	23/07/2022 10:58	File folder	
 Project5210-ApplicationTemplate.sln	16/06/2022 11:55	Visual Studio Solu...	2 KB

TIP: As there may be multiple IDEs installed on a development machine, easiest way is to right-click on the Solution file and select **Open with -> Visual Studio 2022**.



Step 2: Open in Visual Studio and work with the Solution Explorer

After a short while, the project should open in Visual Studio – as usual, we will manage the project using the Solution Explorer view in Visual Studio:

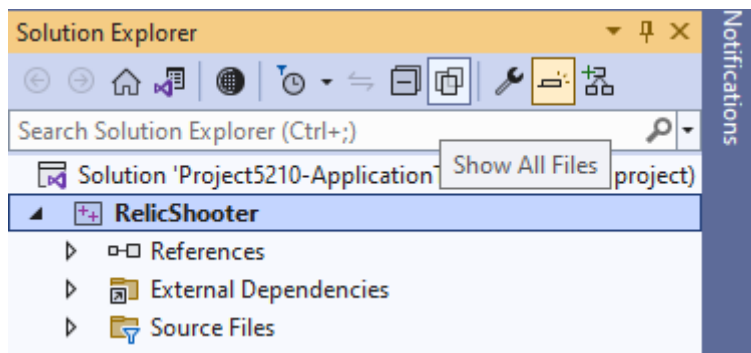


We have one solution (Project5210-ApplicationTemplate) which contains a single C++ Project called RelicShooter. This project contains all the code and assets to build and run the game using our engine.

- You will be using this game engine to complete your assignment so take some time to look through the code to see how it works.

The first thing to understand is that we have two separate views on our code in Visual Studio – a virtual view (or Logical view), which is how we can organize our code, and a Physical View, which is the exact layout of our code on our Hard Drive (e.g., the view of the Files and folders).

You can switch between these using the **Show All Files** option in the menu bar above the solution explorer:



When the option is selected, the icon will be highlighted, as shown below



Highlighted means we are in Physical View, no highlight means we are in Virtual View.

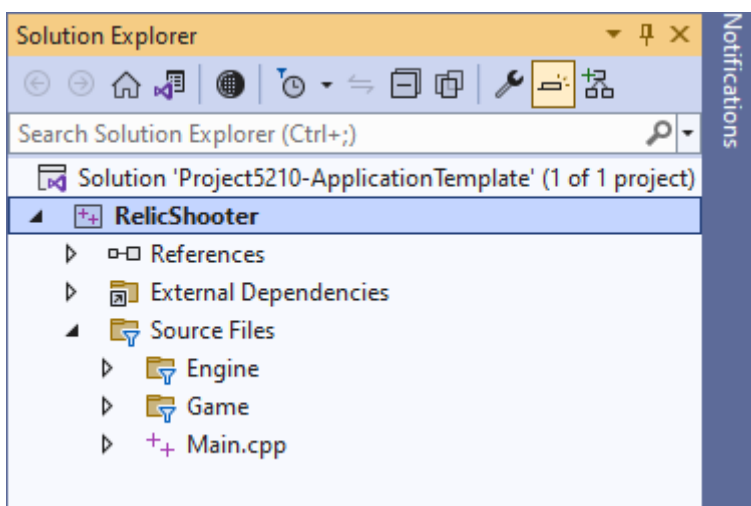
- Test this button, to switch between the two views. Finish by leaving it **not highlighted**.

Step 3: Inspect the Code Base

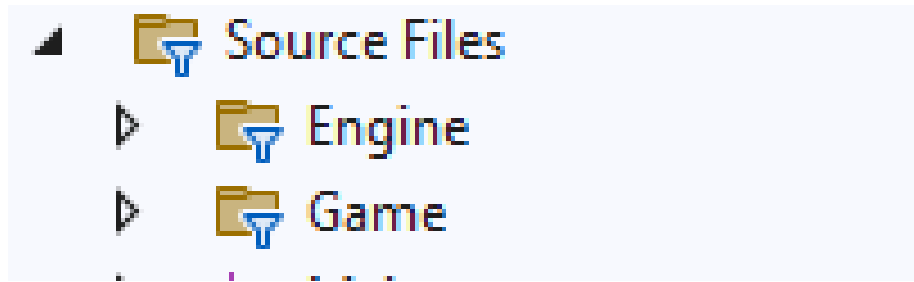
There are two main folders, **Engine** and **Game**.

- All of the Engine code lives in the **Engine** folder (code that can be used by multiple games) and all of the **Game-specific** code will be put into the **Game** folder.

Our current view on the project is known as the **Virtual** view.



- Expand out each of the folders using the Arrow icon by the left-hand side of each Virtual Folder (known as Filters in VS 2022). You can tell the difference between a real folder and a filter by the icon type (the filter looks like a Funnel):



You will see that you have a lot of source code to use as the basis of our game.

- Open the File **Engine/Core/MathHelpers.h**

```

MathHelpers.h
RelicShooter
1 #ifndef _MATH_HELPERS_H
2 #define _MATH_HELPERS_H
3
4 #include <cmath>
5 #include <SFML\Graphics.hpp>
6
7 #define PI 3.14159265359
8
9 namespace MathHelpers
10 {
11     inline float RadToDeg(float angle)
12     {
13         return angle * (float)(180.0 / PI);
14     }
15
16     inline float DegToRad(float angle)
17     {
18         return angle * (float)(PI / 180.0);
19     }
20
21     inline float Length(const sf::Vector2f& vector)
22     {
23         return sqrtf((vector.x * vector.x) + (vector.y * vector.y));
24     }
25 }
  
```

The code has been developed with a specific style (called a Coding Convention) – each game studio will have its own convention – when you work for a company, it's vital you adopt their conventions so that the team has a consistent style.

This code can be built in multiple ways.

In Windows Applications, there are typically:

- two types of Build: **Debug** and **Release**,
- two platforms 32-bit (x86) and 64-bit (x64).

In this case, we are going to initially target a 32-bit application in Debug mode.

To change this, locate the two dropdowns to the left of the Run button on the Toolbar and change the options to Debug and x86 (which means 32-bit), as shown below.



You can close any open code windows.

In this module, we will be using relative references to source code files (putting the relative path in the include folder).

We will also be referencing assets directly based on their relative path, so it is vital that you understand the Physical structure of the code (how it is laid out in folders).

This isn't the only way of doing this in C++, **but it is commonly used in some games libraries – we are going to show you alternative methods in other modules, so you can handle all cases.**

To illustrate this, open **Game/GameData/GameDataManager.cpp**. This is the source code for the GameDataManager class.

You should see the following includes at the top of file:

```
#include "GameDataManager.h"
#include "../Engine/Core/Types.h"
#include "SpritesheetDef.h"

#include <tinycl2.h>
```

We are referencing several header files.

The general rules are as follows:

- If the file has no path, just a name (e.g. "GameDataManager.h"), **then it is in the SAME folder as the file referencing it.**
- If the file has a path, then this is the path **relative to the current file**. The double full-stop means: Go up one directory (to the parent).

Therefore, the Engine folder is two folders up from the current file and then we go into Engine/Core/ and find Types.h

- If the files are in Double Quotes, then they are source code files that are part of the project.
- If the reference is in angled brackets <tinycl2.h> then these are usually **external libraries**, such as the C++ STL (which you used in Level 4, e.g. <iostream>; In this case, the file belongs to <tinycl2.h>.

It is this type of include which we must configure our C++ project to find.

I have already set up these paths for you, which is why the **Libraries** directory has to be in a specific place in our project for the code to work.

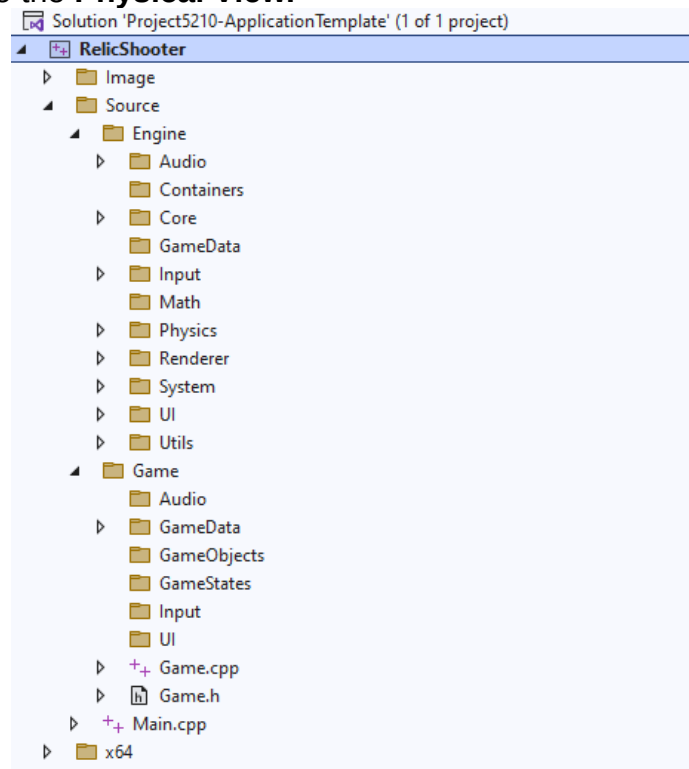
NOT FINDING INCLUDE FILES IS ONE OF THE MOST COMMON SOURCES OF ERROR IN DEVELOPMENT – IT CAUSES LOTS OF ADDITIONAL ERRORS BECAUSE CODE REFERENCING CLASSES, FUNCTIONS ETC WILL NOT WORK, SO VISUAL STUDIO WILL ALSO FLAG THESE AS ERRORS.

Your first port of call should be to check you have the correct includes first. Next, you should use the rules above, to make sure you are referencing the correct file name and have the correct path.

So, how do we know that this is the correct path for GameManager.cpp?

- To see the Physical Structure inside Visual Studio, you will need to press the **show all files icon** in the Solution Explorer, so that it is highlighted.

The View will change to the **Physical View**.



You can see that the code is contained in a folder called **Source** – we have two sub folders, Engine and Game. Unlike our Virtual View – these are actual folders on the Hard Drive.

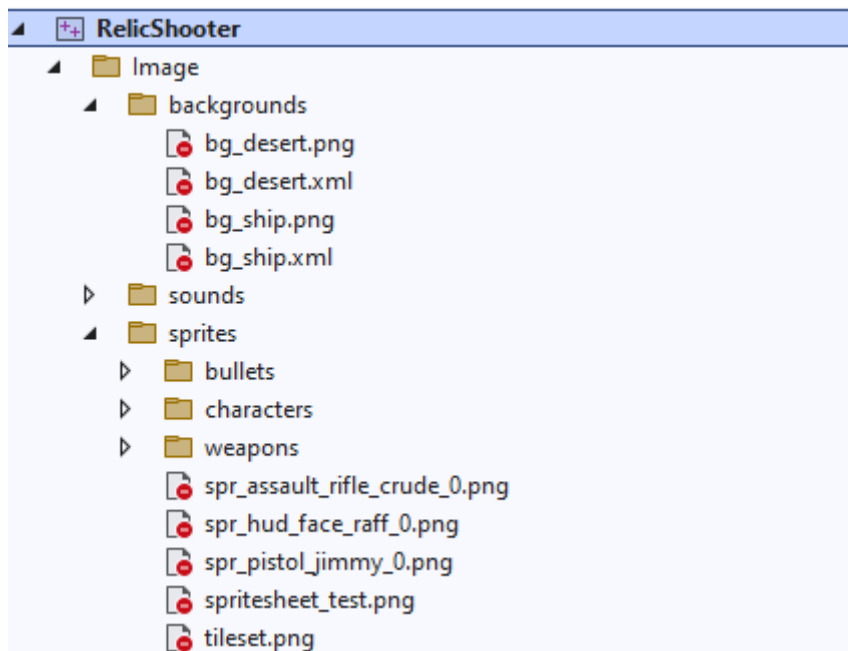
Engine will contain our Game Framework and Game contains our game-specific code.

We will try and avoid any reference to Game in our Engine code (*so that our Engine doesn't depend on our game code*), but our Game will make lots of references to our Engine (*so that we make use of the engine functionality*). We call this Layering – our Game Layer sits above our Engine Layer (*so the Top layer uses code below, but bottom layers never use code above*).

Step 4: Practice Navigating the Code Base

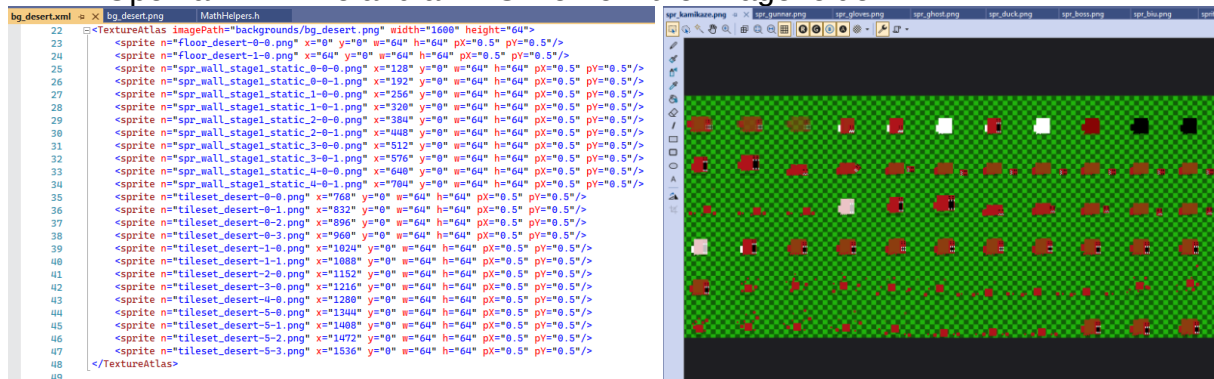
- Try by yourselves to follow the **include path** from **GameDataManager.cpp** to the **Types.h** file so that you understand how to navigate the includes – you will be adding a lot of your own includes as the module progresses.

We also have a folder called **Images** – this contains the assets that we will be loading at run-time. You will always be loading content from this Folder. The Assets folder given to you is a collection of Images to use in your game, but they are not all included by default – you are expected to work out which files you want to use and then migrate them into this Images folder. To begin with, some basic files have been done for you:



We have some background images and some static and animated sprites – we mainly use the PNG image format and XML for data. Visual Studio has in built editors for both.

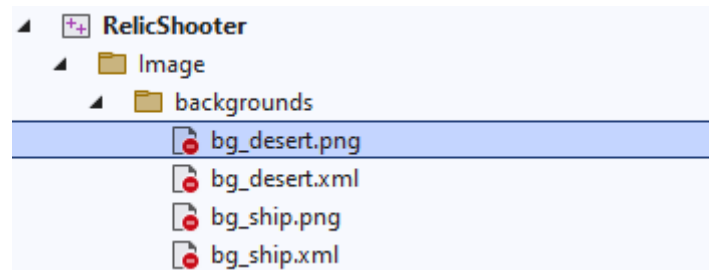
- Open an XML file and a PNG file from the Image folder.



So, you will be copying the assets we need from the Assets folder to the Image folder so that we are loading from Images when the game is running – this is known as Asset Conditioning – we will also be doing some work with the raw Assets so that we have them in a format appropriate for our engine.

Next thing to understand is how Visual Studio decides what needs to be included and compiled in our project.

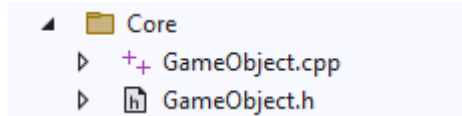
Only aspects that are compiled by Visual Studio (e.g., C++ code) needs to be included in our project. You can see whether a file is included to be compiled, by looking at the icon in the Physical View.



Images and XML Files are not compiled, they are just loaded by the game code, so do NOT need to be included – files that are not included are shown with a Red No-Entry Icon.

Whereas C++ code header and source files (.h and .cpp files) ARE compiled, so we must tell Visual Studio they are included.

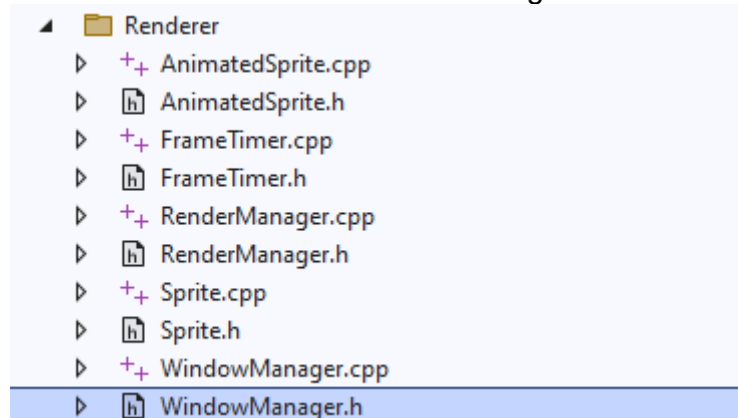
You can tell whether code is included because it appears with the following icons:



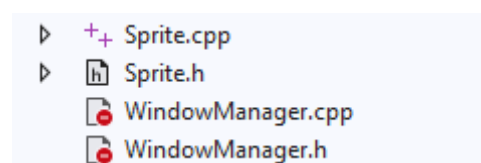
The ++ icon is a .cpp file, the .h icon is a header file.

You can include and exclude code by right clicking on the files.

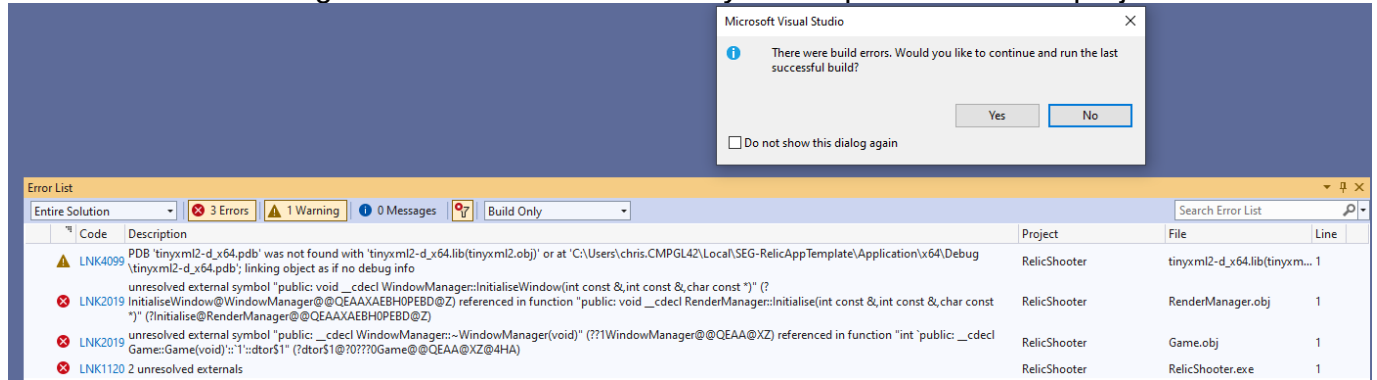
- Find the Header and Source files for the Window Manager:



- Hold Shift whilst left-clicking on the header and source files for WindowManager to select multiple files, then right click and choose 'Exclude from Project'.



The Icon should change to our Excluded icon. Try to compile and run the project.



You should get some errors – they are known as Linker Errors – they happen when a piece of code is being called but has not been compiled – in this case, when we try and create a Window Manager and Initialise it, we can no longer do this because the class has not been compiled.

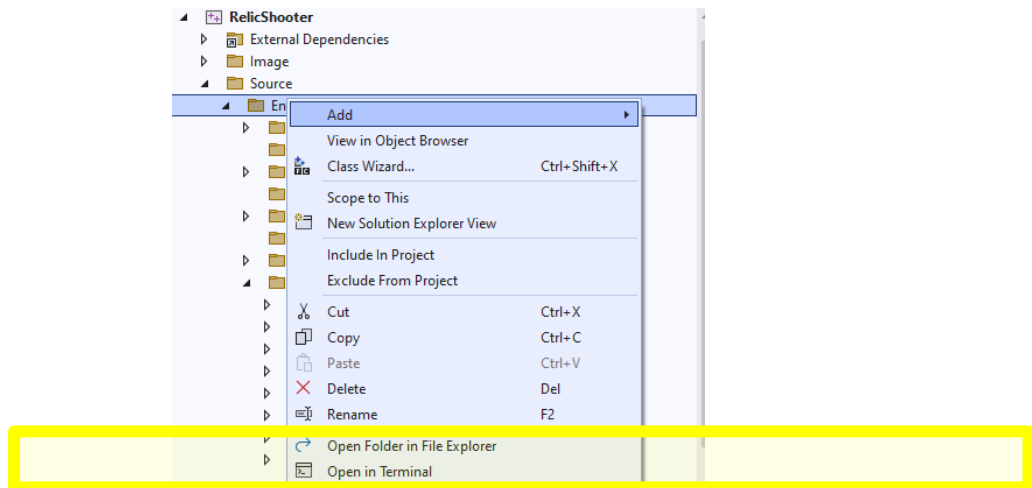
- Select the Two files again and right click, this time choosing 'Include in project'.
- Make sure that the code compiles correctly and runs.

So, whenever you add new code to the project – if you want to compile it, it MUST be included. If you have code that is incomplete and don't want it to be compiled, you MUST exclude it.

Step 5: Accessing the Files on your Machine

Often, you will want to move files around using the Operating System. To easily find them, Visual Studio can help.

- You can right click on a File or Folder in the Physical View and select 'Open Folder in File Explorer'



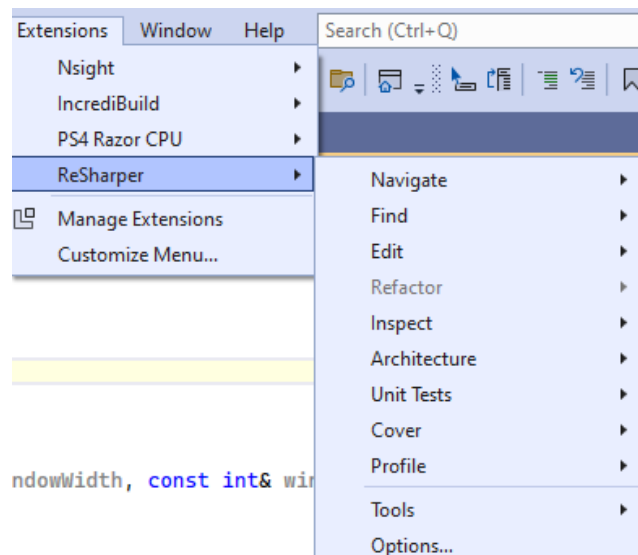
- Have a look over the Project using Windows Explorer, so you are sure that the view we are seeing is actually how the files are laid out on the C drive.

Step 5: Making the Most of Visual Studio!

The previous few tasks have focused on the essentials of working in Visual Studio, but we also have another tool that can help us – ReSharper C++. ReSharper is a Plugin/Extension to Visual Studio that is designed to make developing using C++ much easier. It is developed by a company called JetBrains, who make lots of development tools used in the Games Industry (such as an IDE called Rider which is used with Unreal and Unity).

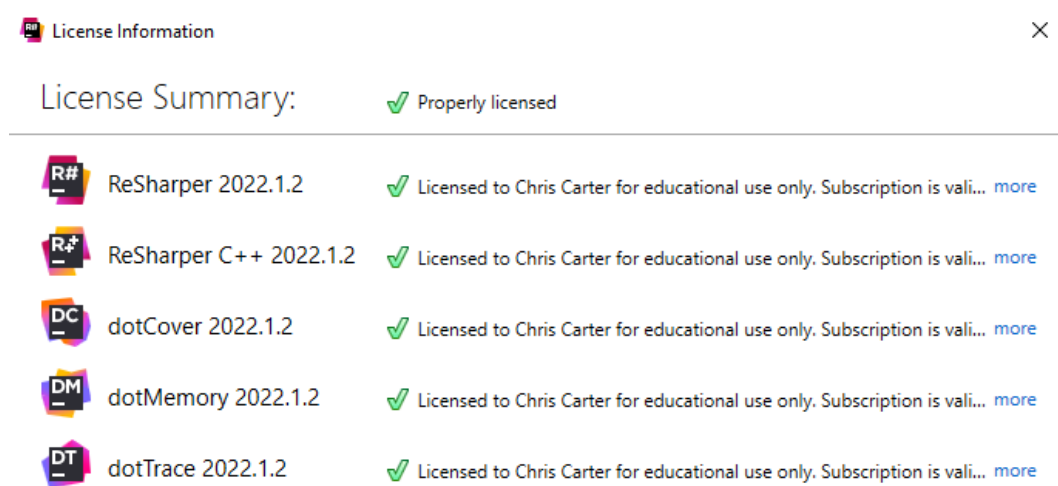
ReSharper is a set of additional options designed to make Software Engineering in both C++ and C# easier. We have licenses for ReSharper in the University and on Canvas, there are details on how you can get it at home.

My Advice – use ReSharper whenever you can! You can get to it via the Extension Menu:



To make sure it is enabled correctly in the Labs, go to:

Extensions -> ReSharper -> Help -> License Information



In the labs, we have a License server that will validate the license. This should be completed for you with the details of our Server.

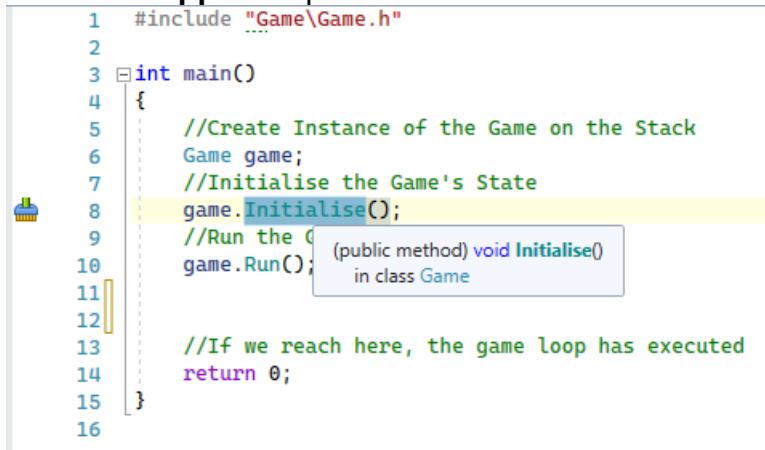
At home, you will need your own Jet Brains account, which you can create on their website using your LJMU e-mail address to get your own copy of ReSharper for home. We have a deal with JetBrains to approve LJMU Games Students with an LJMU e-mail address automatically, which makes getting a license easier. For more details, have a look on Canvas under the [Working from Home](#) section.

Here is an example of licensing it – the Spanner icon allows you to log in.

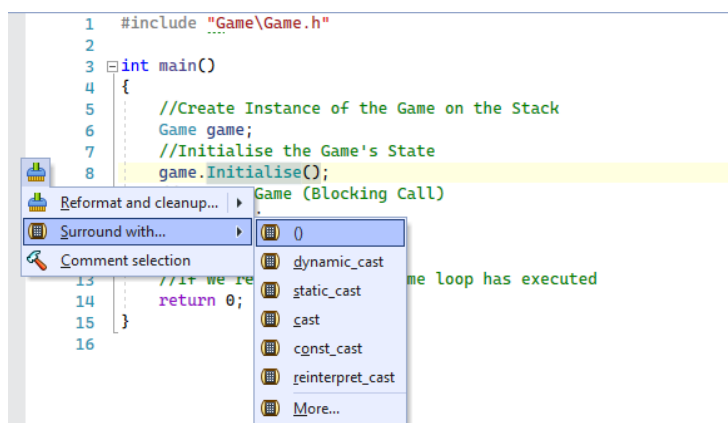


Let's conclude this task will a look at a couple of features of ReSharper – the module leader will illustrate them live during the sessions for you as we go through the module.

- Find **Main.cpp** and open it. Double click on the Word 'Initialise'.

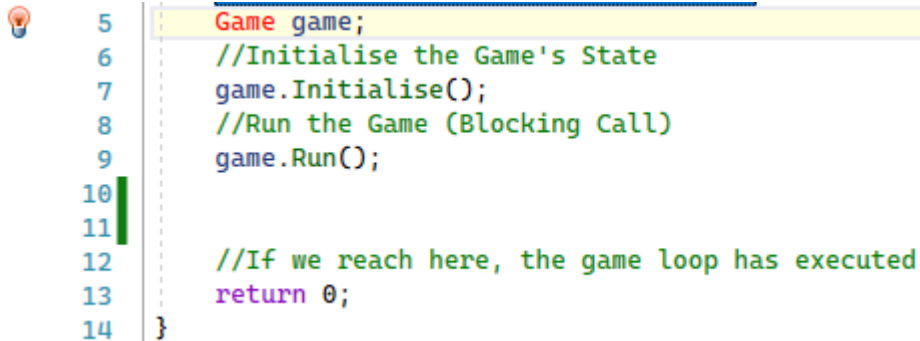


We can tell ReSharper is running, because we will get other icons (in this case the Brush icon which helps us clean up code). You can then click this icon to get contextual help with your code – actions to help make your code better:



If you don't see anything, get one of the module team to help you!

- If ReSharper is present, you can test one of its main features by deleting the first line (Line 1) – the include statement.

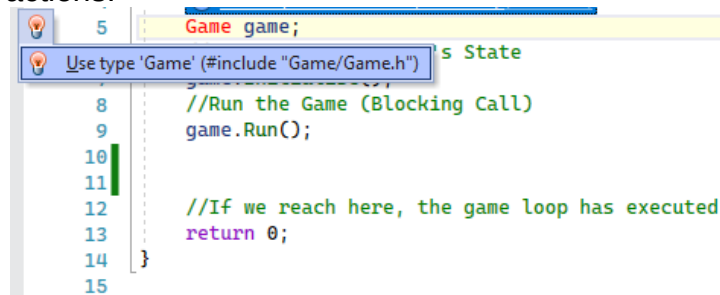


```

5  Game game;
6  //Initialise the Game's State
7  game.Initialise();
8  //Run the Game (Blocking Call)
9  game.Run();
10
11
12 //If we reach here, the game loop has executed
13 return 0;
14 }

```

- Because we have got rid of the include, Main doesn't know about our class Game anymore – but notice there is a light bulb. Click on this light bulb to get the ReSharper suggested actions:



```

5  Game game;
6  //Initialise the Game's State
7  game.Initialise();
8  //Run the Game (Blocking Call)
9  game.Run();
10
11
12 //If we reach here, the game loop has executed
13 return 0;
14 }
15

```

- In this case, the suggested action is to include Game.h (which we know is right, as we deleted this working code!). Click the action, and ReSharper will generate the code for you. So, ReSharper can automatically find the right class for you and fill in the include – this is just one example – there are dozens of features that it will help with.

<https://www.jetbrains.com/resharper-cpp/documentation/resharper-cpp-vs-visual-studio.html> has a table with links to each of the key features it adds.

The next few tasks (and future tutorials) will be less-specifically directed – I want to see how much you have remembered/are yet to learn properly about C++.

To help you, I advise looking at the **learncpp.com** website as a reference – the link to this is on Canvas.

Support - Online C++ Learning Materials			✓	+	⋮
📄	Info: Furthering your C++ Development Skills		✓		⋮
📄	Info: C++ Programming videos		✓		⋮
🔗	Learn CPP Portal		✓		⋮
🔗	ISO C++ FAQ - Written by Bjarne Stroustrup		✓		⋮
🔗	TIO - C++ Online		✓		⋮
🔗	Godbolt Compiler Explorer		✓		⋮

- Click the Learn CPP portal link.

Learn CPP Portal

Inheritance

Chapter 17

17.1

Introduction to inheritance

17.2

Basic inheritance in C++

17.3

Order of construction of derived classes

17.4

Constructors and initialization of derived classes

17.5

Inheritance and access specifiers

17.6

Adding new functionality to a derived class

17.7

Calling inherited functions and overriding behavior

17.8

Hiding inherited functionality

17.9

Multiple inheritance

17.x

Chapter 17 comprehensive quiz

Virtual Functions

Chapter 18

18.1

Pointers and references to the base class of derived objects

18.2

Virtual functions and polymorphism

18.3

The override and final specifiers, and covariant return types

18.4

Virtual destructors, virtual assignment, and overriding virtualization

18.5

Early binding and late binding

18.6

The virtual table

18.7

Pure virtual functions, abstract base classes, and interface classes

18.8

Virtual base classes

18.9

Object slicing

18.10

Dynamic casting

18.11

Printing inherited classes using operator<<

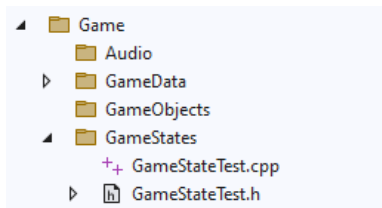
Learn CPP has an excellent set of guides on most the C++ syntax you will use on this module – I recommend reading through the start of Chapter 17 before starting the next few tasks!

Task 3: Create a blank Game State

Step 1: Create a new class

Create a new C++ class called **GameStateTest** in the **Game->GameStates** folder. You can do this via the Class Wizard, or by add new files called **GameStateTest.h** and **GameStateTest.cpp** to this folder.

NOTE: The class wizard often generates files in the root of the project – you will need to drag these files to the correct folder if you use this approach:



This class should inherit from the **GameState** class. This is done in the header file.

You will need to include **GameState.h** to be able to inherit from **GameState**.

To work out the syntax for inheritance, have a look at some of the other classes (e.g. **Engine -> UI -> Text.h** – you can see that **Text** inherits from **sf::Text** – note the use of the **public** keyword!).

Step 2: Add the required functions

As you have inherited from the **GameState** class you must have the following public functions:

- A constructor
- A destructor
- A function called **Initialise()** with the return type **void**.
- A function called **Update()** with the return type **void**.
- A function called **GetPlayer()** with the return type **GameObject** pointer. This function should just return **nullptr** for now (**nullptr** is the modern way of using **NULL** in C++).

We will discuss why these functions are required when we cover virtual functions. You should also use the **override** keyword with these functions.

Step 3: Call Update()

In the **Update()** function, call **GameState::Update()**; This will call the parent class' version of the update function.

Task 4: Add the Game State to the game

Now that we have a blank game state, we need to add it to the game so that it will run.

Step 1: Create an Instance of the GameStateTest class

Open up **Game.cpp** and locate the `Initialise()` function.

At the bottom of the function, create a variable that is a pointer to a `GameStateTest` class and initialise it using the **new** keyword.

Remember syntax for creating a object is

`Class* object = new Class();` where `Class` is the name of the class you want to create.

Step 2: Queue the GameStateTest with the game state manager

Call `QueueGameState` on the `m_GameStateManager` object and pass in the pointer to the `GameStateTest` object that you created in the previous step.

This will cause the Game State Manager to Run the new game state until it is replaced or removed from the game state manager.

When the game state is pushed on to the game state manager, the `Initialise()` function will be called once, and then the `Update()` function will be called every game frame until the game state is removed.

Task 5: Create a Text Game Object

We have a blank game state that is running every frame, however, to see this in action we need to create a game object and add it to the game state.

Step 1: Create a new class

Create a new class called **GameObjectWithText** in the Game->GameObjects folder.

This class should inherit from the GameObject class.

You will need to include GameObject.h to be able to inherit from GameObject.

HINT: Try using Resharper to Help you with this. Once you have created the class header and source file, have a look at the generate code options in the Resharper menu.

Step 2: Add the required functions

As you have inherited from the GameObject class you must have the following public functions:

- A constructor
- A destructor
- A function called Initialise() with the return type **void**.
- A function called Update() with the return type **void**.
- A function called GetPosition() with the return type **void** and two **floating point (float) reference parameters** called x and y.
- A function called GetPosition() with the return type **sf::Vector2f**
- A function called SetPosition() with two floating point parameters called x and y and return type **void**.
- A function called SetPosition() with one **sf::Vector2f** parameter and return type **void**

Have a look at the **GameObject.h** file to see an example of the definition of each of these functions – you are going to be implementing these functions.

We will discuss why these functions are required when we cover virtual functions.

- You should also add a member variable called **m_Text** with the type Text.
- You will need to include **Text.h from the Engine (not SFML sf::Text!)** to create the m_Text member variable.

Step 3: Initialise the Text member variable

In the `Initialise()` function, call `Initialise` on the `m_Text` member variable, passing in “fonts/8-BIT WONDER.TTF” and “Hello World” as the parameters.

This font is going to be found relative to the `Image` folder, so find the 8-Bit Wonder font file in the `Assets` folder and make a new folder inside the project’s **Image** folder called `fonts` and copy the font file (a true type font or ttf file) into this folder.

Step 4: Fill in the `GetPosition` and `SetPosition` functions

The **`GetPosition`** function taking the two integer references, should set the reference parameters to the position of the `m_Text` member variable.

You can do this by calling `getPosition().x` and `getPosition().y` on the `m_Text` member variable. This is using the SFML Text API (as `Text` inherits from `sf::Text`).

The **`GetPosition`** function which returns a `sf::Vector2f` should return the `getPosition()` function of `m_Text`.

The **`SetPosition`** function which takes two floats should use `m_Text`’s `setPosition` function.

The version which takes an `sf::Vector2f` should also use this same function (there are different versions of the same function – this is called overloading) – look up overloading on learncpp.com if you are struggling.

Task 6: Add a Text Game Object to the GameStateTest

We now have a text Game Object that we can add to the GameStateTest class to see that it is working.

Step 1: Create an instance of the GameObjectWithText class

Open the GameStateTest.cpp file and locate the Initialise() function. Create a variable that is a pointer to a GameObjectWithText class and initialise it using the new keyword.

Step 2: Initialise the GameObjectWithText

Call the Initialise function on the pointer variable that you created in the previous step.

Step 3: Add the GameObjectWithText Instance to the GameObject list

Call push_back() on the m_GameObjects member variable, passing in the pointer to the GameObjectWithText that you have created.

When you run the game, you should end up with something similar to the following:

SEG - Relic Shooter

HELLO WORLD

Additional Task: Make the *GameObjectWithText* generic

You don't want to write a new class every time you want to put some text on the screen or change the font. We need a better level of Abstraction for this class.

Make the *GameObjectWithText* class more generic and reusable so that you can:

- specify the font
- specify the colour
- specify the size
- specify the text (you can use `char*` or the C++ string class to do this – see what the function `c_str()` does!).

<https://cplusplus.com/reference/string/string/>

Making it generic means adding data to store the font, colour, size and text in the class and then creating Get and Set functions (like `GetPosition` and `SetPosition`) to modify this data.

It is also worth pointing out that you are free to make changes to the Engine code as well – in this case, you may want to consider modifying the *Text* class – in particular, changing the signature and parameters that are passed to the *Initialise* function (currently it is just the font and string).

You can then update the *Initialise* function of the Engine's *Text* class so that it uses this data, instead of hardcoding the information (e.g. instead of hard coding the font name so its always , it should use a class member variable).

Again, speak to the module team if you are struggling with this. Use the resources on Canvas and have a look at the code within the code base (e.g., the other classes) to help you with this!

Here is an example of how we might end up using this Generic Class (it's fine if your function names etc. are different!) and the output achieved.

```
/**
 * \brief Called when the State is first loaded.
 * Sets up the Initial State of the Screen.
 */
void GameStateTest::Initialise()
{
    //Add a Text Object
    GameObjectWithText* tText = new GameObjectWithText();
    tText->SetCharSize(psize: 50);
    tText->SetDisplayText(ptext: "Generic Text");
    tText->SetColour(pcolour: sf::Color::Blue);
    tText->SetFontFile(pfont: "fonts/wendy.ttf");
    tText->SetPosition(ppos: sf::Vector2f(x: 100.f, y: 100.f));

    //Add it to the Renderer
    tText->Initialise();

    //Add it to the Game Objects
    this->m_GameObjects.push_back(tText);
}
```



Notice the use of our `SetPosition` function.